KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department: Computer Science and Engineering

Project Report

Project Title: Multi-Modal Control of Niryo Ned Robotic Arm

Course Title: Embedded Systems and Internet of Things Laboratory

Course No: CSE 3106

Submission Date: 14 July 2026

Submitted To:	Submitted By:
Dr. Muhammad Sheikh Sadi Professor & Md. Repon Islam Assistant Professor Department of Computer Science & Engineering Khulna University of Engineering & Technology	Nurul Absar Shadhik Roll: 2207065 Omar Faruk Rakin Roll: 2207082 Nafis Ahammad Roll: 2207084 Md. Farhaduzzaman Rume Roll: 2207080

Project Description:

Our project develops a multimodal control system for the Niryo Ned, a 6-axis collaborative robotic arm, by integrating three independent control methods: autonomous vision-based operation, wireless voice control, and real-time hand-gesture control. Each subsystem runs on the hardware platform best suited to its processing requirements while controlling the same robot and calibrated vision work-space.

The system combines computer vision, speech recognition, gesture recognition, and wireless communication to enable autonomous object sorting, hands-free voice operation, and intuitive real-time teleoperation. By supporting multiple human-robot interaction methods on a single robotic platform, the project demonstrates a flexible, efficient and user-friendly approach suitable for industrial automation, research and educational applications.

Features:

i) Fully autonomous object detection, pick-up and color based sorting (RED/GREEN/BLUE bins),

ii) Wireless robot control from a separate Raspberry Pi - no physical connection to the robot needed,

iii) Voice command recognition with two selectable speech-to-text engines (cloud-based Google STT and offline faster whisper),

iv) Spoken audio feedback to the user (text-to speech) confirming each action,

v) Headless, unattended operation of the voice-control Pi via a systemd background service,

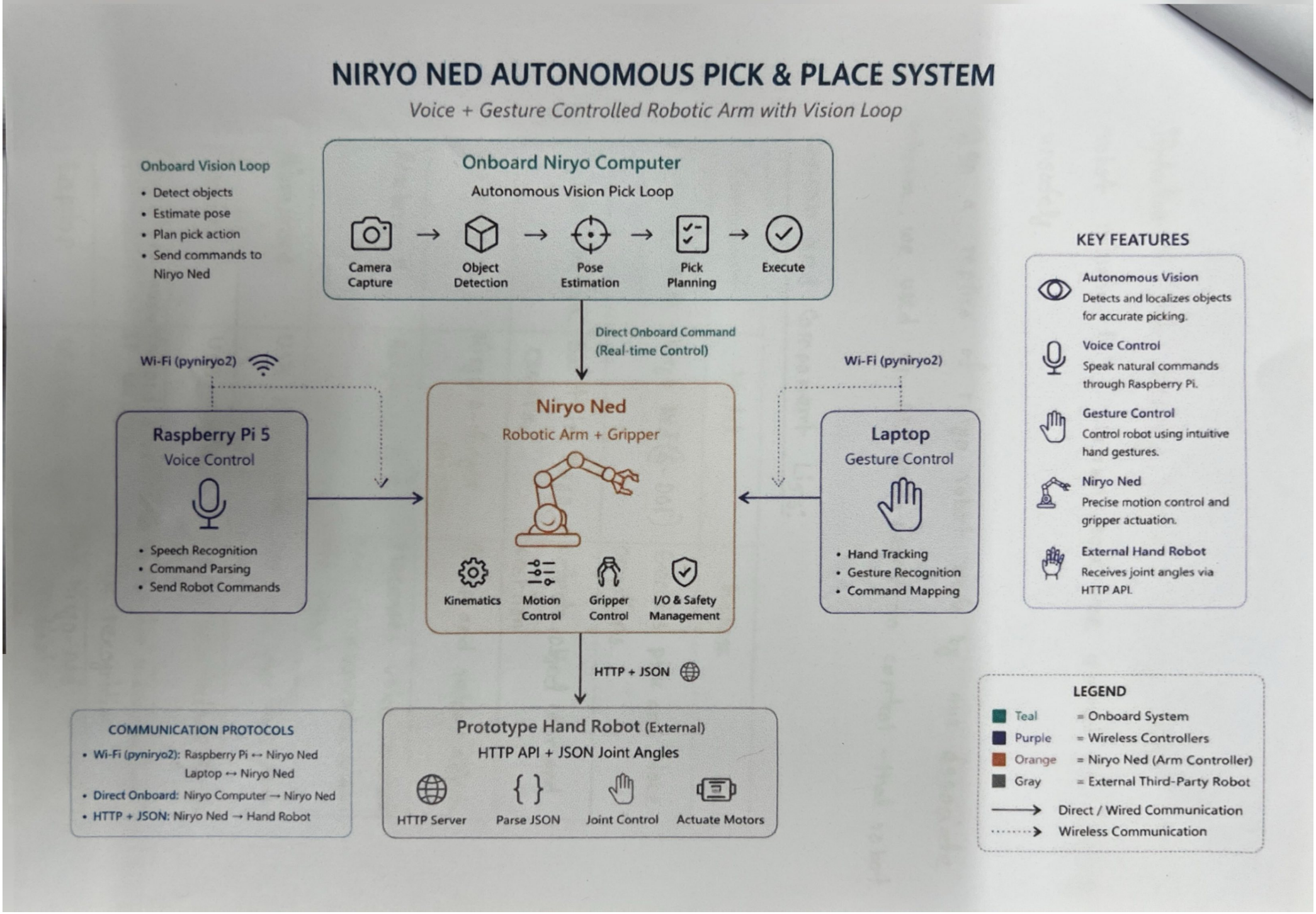
vi) Real-time hand-gesture recognition (pinch, open hand, point, hand-raise) mapped to robot actions.

vii) Wireless video streaming from a phone camera to a laptop for gesture control,

viii) Robust state-machine control logic that prevents gesture/voice misfires while the robot is mid-motion

NIRYO NED AUTONOMOUS PICK & PLACE SYSTEM

Voice + Gesture Controlled Robotic Arm with Vision Loop



ix) Automatic safe recovery behavior: on any error, the robot returns to a known safe pose instead of stopping unsafely.

x) On a replica of niryo robot made by our associate team, we used an API end-point to control that robot.

Sensors and Component Lists:

Component	Model	Purpose
Robotic Arm	Niryo Ned (6-DOF)	Executes pick and place operations.
Vision Camera	Built in Niryo camera	Detects object color and position
Gripper	Niryo 2-finger gripper	Grasp and release objects
Raspberry Pi	Raspberry Pi 5	Processes voice commands and communicates with the robot
Microphone	USB microphone	Captures user voice comm-ands
Speaker	USB speaker	Provides voice feedback to the user
Smartphone Camera	Android phone with IP webcam	Stream live video for hand gesture recognition
Laptop	Windows Laptop	Run Niryo-ned simulator. Run mediapipe and gesture recognition algorithm

Wireless Network	WiFi (LAN)	Enable communication between all devices.

Software Specification:

Software/Application	Version	Purpose
Python	Python 2.7 and Python 3	Main programming language for robot control
Niryo studio	Niryo-ned	Robot calibration, workspace creation and testing
Niryo ROS API	Built-in robot API	control the robot directly to the on board computer
pynirgo2	Python library	Sends commands to the robot over wi-fi
Opencv (cv2)	Computer vision library	Image processing and camera handling
Mediapipe	Hand Landmarkers	Detects and tracks hand gestures.
Speech_recognition	Python library	Convert voice to text
Pyttsx3	text to speech library	Generates voice feedback for the user

IP webcam	Android application	Streams smartphone camera video to the laptop.
Raspberry Pi OS/ Windows 11	Operating system	Run the control application on Raspberry Pi and laptop.

Implementation:

Initial Phase 1: (Autonomous Vision-Based Pick and sort)

i) At first, we downloaded and installed the Nirjo Ned Software suite (Nirjo Studio) on the host computer.

ii) After that, we powered on the Nirjo Ned and conned our computer to the robot's own wi-fi hotspot.

iii) we verified the connection inside Nirjo studio, confirming the Robot's IP address is reachable before doing any further work,

iv) After doing this, we created a new vision workspace named work_2k22 corresponding to the physical platform in front of the robot where objects will be placed.

v) Calibrated the workspace by manually guiding the robot's camera to the four corner points of the platform, one at a time, so the robot can build an accurate co-ordinate mapping between the camera image and real world positions.

vi) The calibration step is what allows vision_pick() later to correctly translate a detected object's pixel location into a real arm-movement pose.

vii) Using the Robot's learning mode (manually moving the arm by hand and recording the pose); three types of pose were fixed:

→ Observation pose (obs_pose): Positioned so the camera has a clear, consistent view of all four calibrated workspace corners.

→ Three drop poses - one each for red, green and blue objects, positioned above the corresponding sorting bins.

→ Home pose (home_pose) → a neutral resting position the arm returns to after each drop, before

going back to observe again,

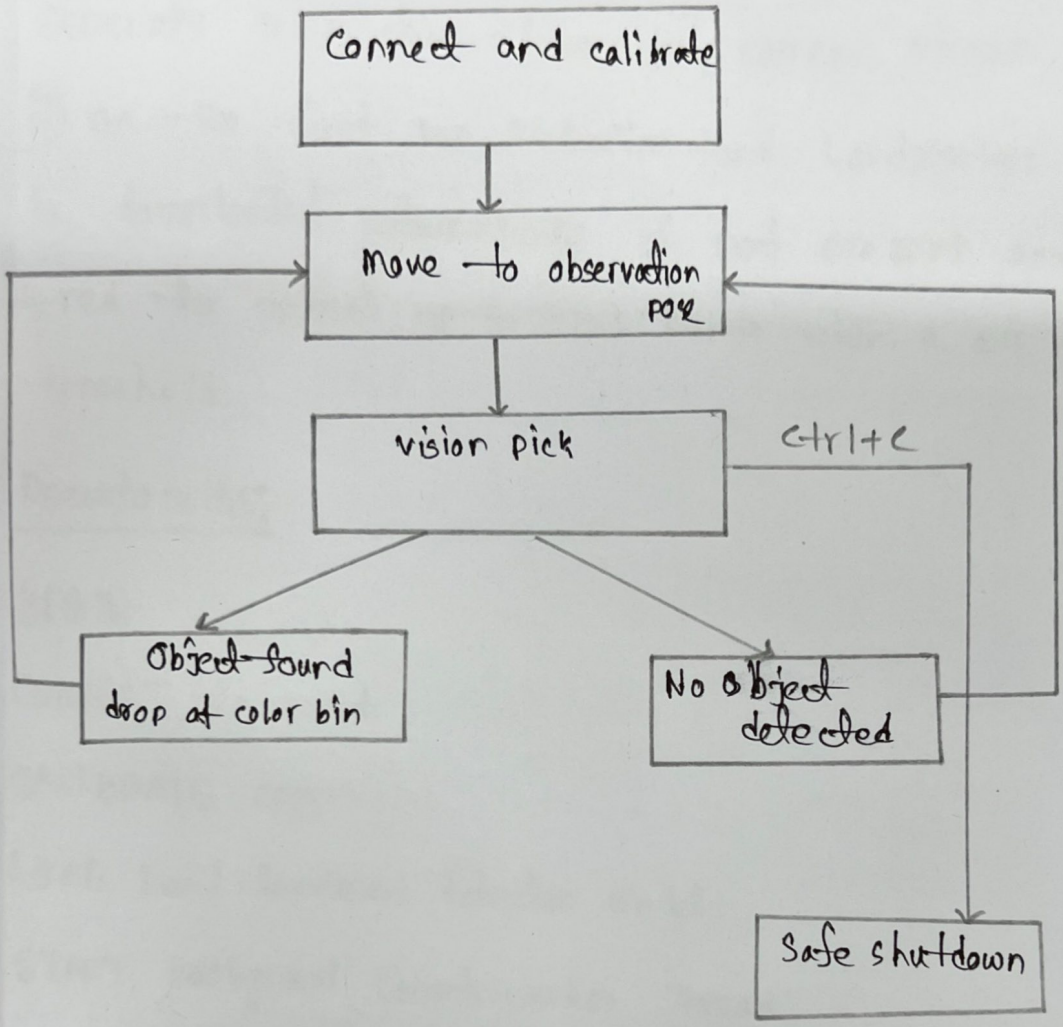
After doing all of these, we implemented a code for doing the work.

Pseudocode:

```
BEGIN
CONNECT to robot at fixed IP
if connection fails:
    print error
    exit program
define workspace name="work_2422"
define observation_pose, home_pose
define drop_pose for red blue, green
CLOSE GRIPPER
function go_to_observation():
    move arm to observation_pose
    wait briefly
CALIBRATE arm automatically
CALL_go_to_observation()
LOOP forever:
 found, shape, color = DETECT and pick object in workspace
```

```
If found:
    print detected shape and color
    close gripper
    wait briefly
    If color matches in a known drop_pose:
        move arm to drop_pose [color]
        wait briefly
        Open gripper
        wait
        move arm to home_pose
    CALL go_to_observation()
    ELSE:
        print "No object found"
        CALL go_to_observation()
    WAIT 1 second
    close gripper
    IF user presses ctrl+c:
        break loop
ON exit (normal or interrupted):
    CALL go_to_observation()
    DISCONNECT from robot safely
    print ("safe shutdown")
END
```

Visualization:



Initial Phase-2: (Gesture control)

i) We run a smartphone camera streaming app (IP webcam) and exposes the live feed over wifi, so the camera can be freely positioned near the hand instead of being tied to the laptop.

ii) The laptop connects to the robot via same network seperate connection from the camera stream.

iii) on the first run, MediaPipe Hand Landmarker model is downloaded automatically if not present and configured to detect up to two hands with a 0.7 confidence threshold.

Pseudocode:

```
BEGIN
CONNECT to robot
CALIBRATE arm
LOAD hand landmark detection model
START background robot_worker Thread:
    MOVE to observation pose, OPEN GRIPPER, STATE=OBSERVING
    LOOP:
        WAIT for next command in queue
        if command is 'observe': MOVE to observation pose, open gripper, state= observing
        if command is 'pick':
            STATE = PICKING
            found= DETECT and GRASP objects
            IF FOUND: STATE = HOLDING
```

ELSE: MOVE to observation pose, STATE = observing

if command is "drop_pose": Move to drop_pose, STATE = READY_to_DROP

if command is "drop": OPEN gripper, STATE = idle

ON any error: move to observation pose, STATE = OBSERVING

OPEN camera stream from phone IP Address

LOOP forever (main video loop):

frame = GRAB latest frame from stream

if frame missing: CONTINUE

EVERY 3rd frame:

landmarks = DETECT RIGHT HAND only

if hand found: SAVE landmarks, timestamp

ELSE IF landmarks stale for > 0.5s: clear landmarks

if landmarks available:

pinch_dist = DISTANCE (thumb_tip, index_tip)

angle = ANGLE (wrist → middle_finger_tip)

pinching = pinch_dist < 0.04

open_hand = pinch_dist > 0.08

hand_up = fingers raised well above wrist AND OPEN_hand

pointing = angle within ± 40° of horizontal

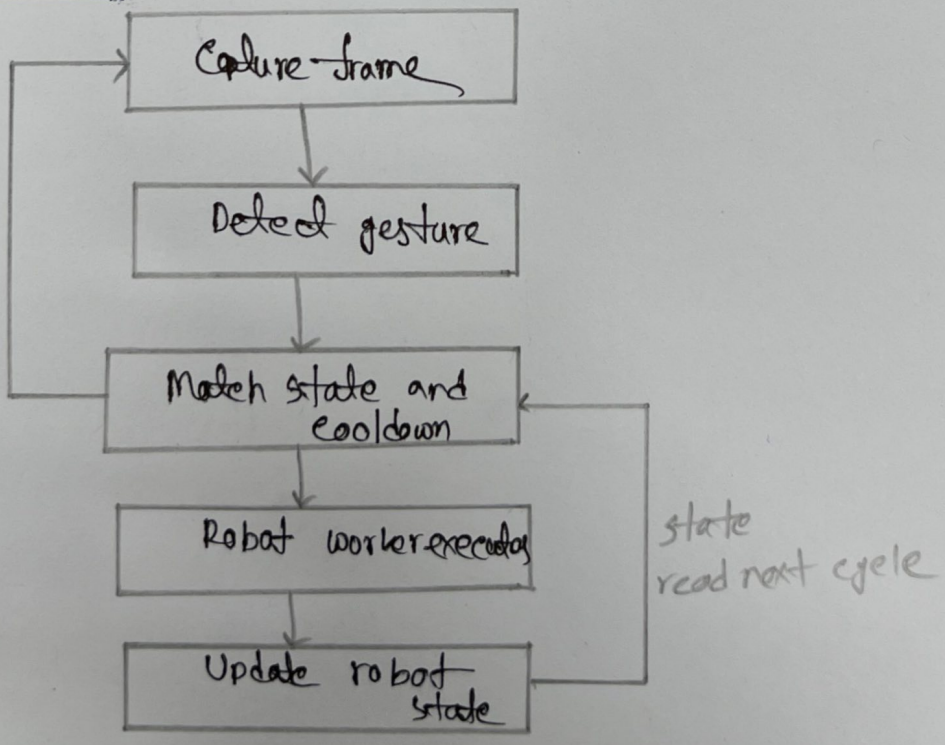
current_state = READ robot state

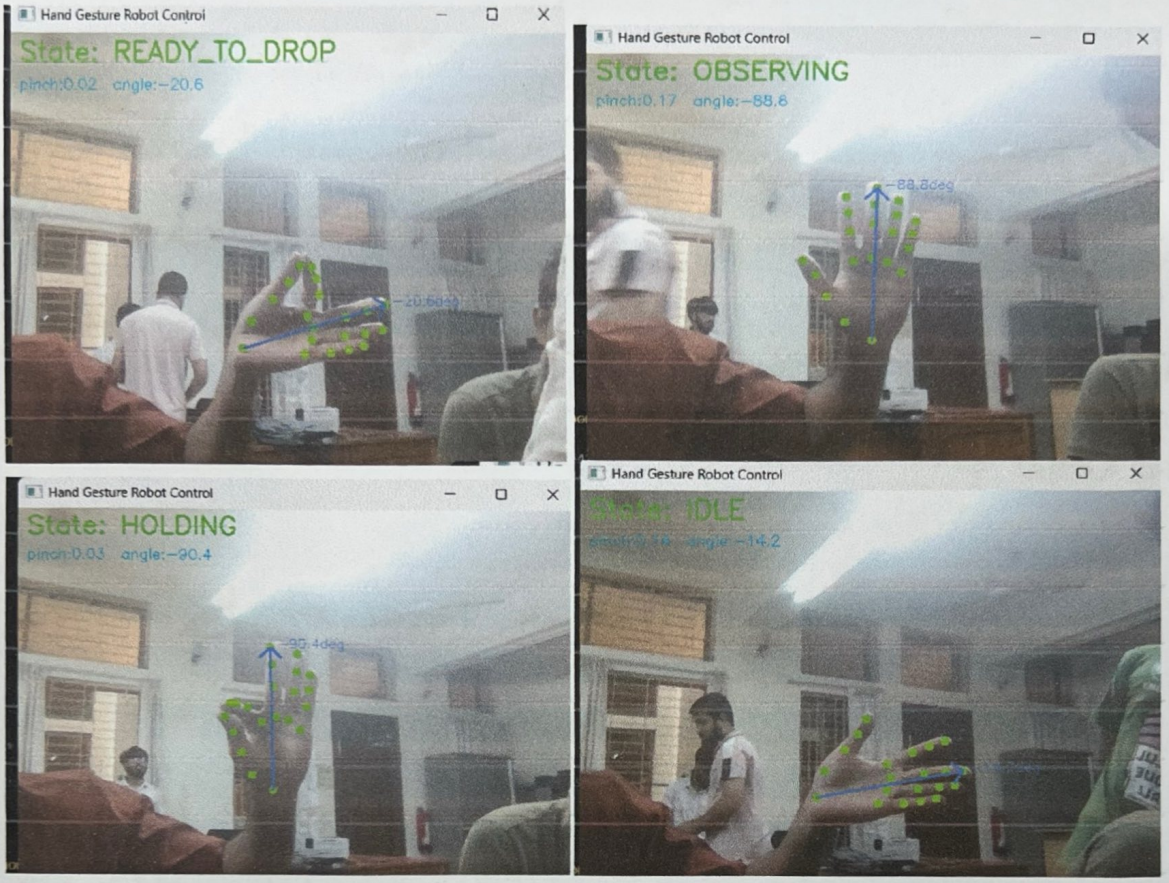
```
if cooldown has passed:
    if current_state= picking: DO nothing (busy)
    ELIF pinching AND state is IDLE/OBSERVING: SEND 'PICK'
    ELIF painting and state is holding: send "drop_pose"
    ELIF Open_hand AND state is READY_TO_DROP: SEND "DROP"
    ELIF hand_up AND state not in (HOLDING,Picking): send "observe"

DRAW landmarks, gesture values, and state on screen
Show video window
if 'q' pressed: break

ON Exit:
    RELEASE CAMERA, close detector
    stop robot_worker thread
END
```

Visualization:





orintial

Phase - 3: (Voice command)

i) Connected the laptop to the same wifi network as the Nigro Ned,

ii) Configured the robot's IP address in the control program,

iii) Established a wireless connection between the laptop and the robot,

iv) Initialized the robot control program,

v) Started the voice control application.

vi) Verified successful communication between the laptop and the robot.

vii) Began listening for voice commands

viii) Executed robot actions based on recognized commands.

Pseudocode:

```
BEGIN
CONNECT to Robot
SPEAK "Initializing robot"
CALIBRATE arm, update tool
MOVE to observation pose
SPEAK "READY, say a command!"
LOOP forever:
    command = LISTEN and transcribe (Google or whisper)
```

if command is empty:
    continue
if command contain exit/stop/shutdown/quit:
    SPEAK "shutting down"
    BREAK
ELIF command mentions check/what/see/detect:
    SPEAK detected object's shape and color
ELIF command mentions color:
    SPEAK detected object's color only
ELIF command mentions sort:
    DETECT and pick object
ELIF command mentions observe/home:
    return to observation pose
    speak "Ready"
ELIF command mentions pick/grab/take
    DETECT and pick object
    speak success or failure
ELIF command mentions "drop <color>":
    move to that drop color drop pose
    OPEN gripper
ELIF command mention "drop/place/release":
    move to default drop pose, open gripper
ELSE:
    SPEAK "command not recognized"

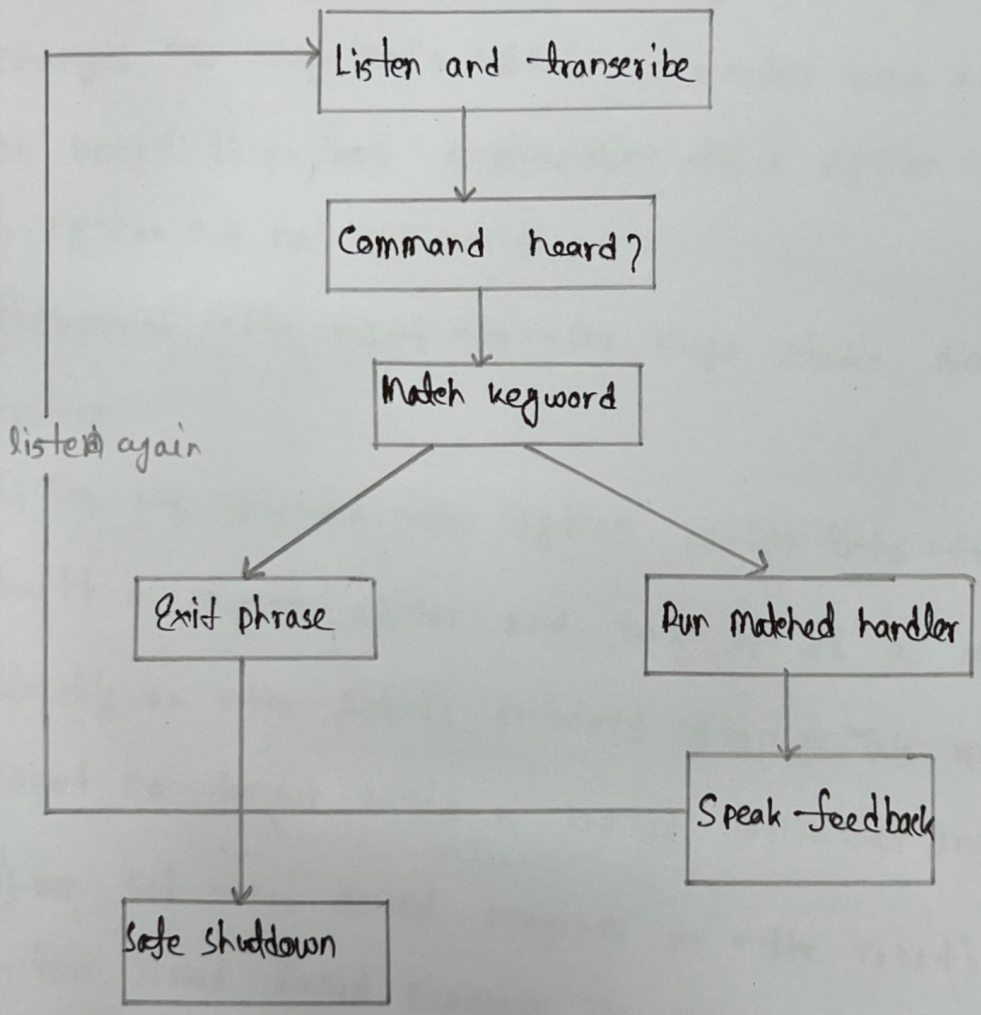
ON EXIT (normal, interrupted, or error):

return to observation pose

SPEAK "DISCONNECTED"

END

Visualization:



Final Phase-1: (VisionPick loop)

i) On the end, we running this program on the Nirjo Ned's on board Raspberry Pi.

ii) For this, we need to change code. Everything remain same but for version mismatching something should be changed. The Nirjo Ned's built in computer runs an older on board Linux/ROS environment whose python interpreter is Python 2.7 not Python 3.

iii) Connect the robot to the Nirjo studio same like before,

iv) The we uploaded the python script into the robot's built in program editor and save it as a program stored directly on the robot's onboard storage. This means the robot no longer needs a laptop connected once it's saved.

v) we set this saved program as the robot's autorun/button linked linked program. The Nirjo red has a single physical action button on top of its base; Nirjo studio lets you designate which save program that button triggers.

vi) we pressed the physical button on the robot.

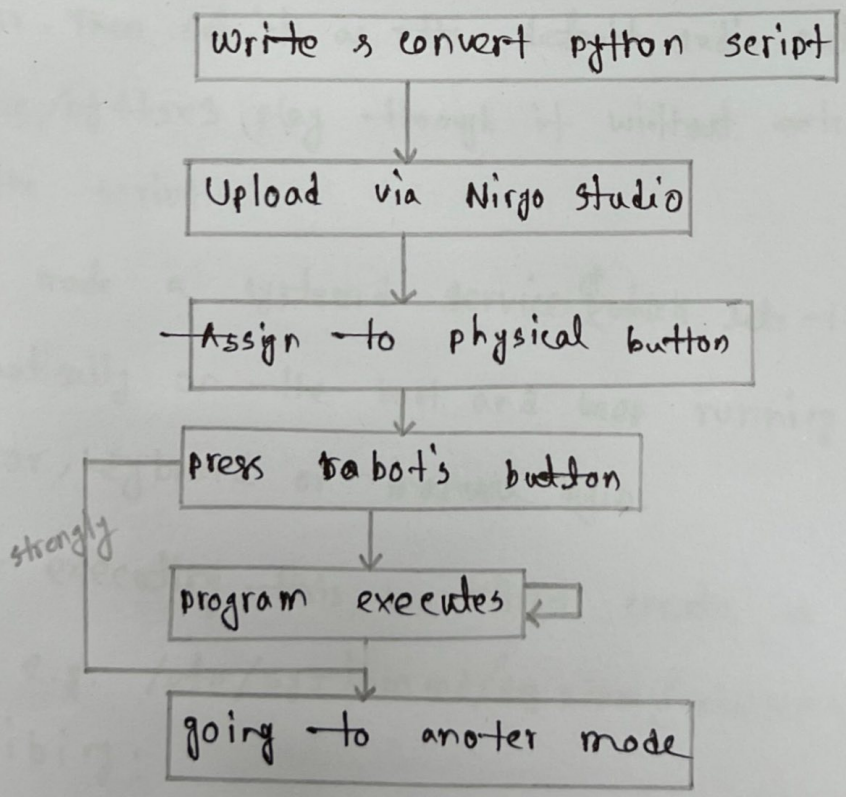
This tells the onboard computer to load and execute the selected saved program immediately - no external device laptop, or network connection is needed at that point, since everything (code + robot control) now lives entirely on the robot itself.

vii) The vision pick loop then executes exactly like before, but fully self-contained on the robot.

Pseudocode:

Same like initial phase. Just converted it python 2.7. Entire logic remained same.

Visualization:



Final Phase-2: (Gesture control)

This phase is similar to initial phase. We don't modify anything for final phase. It is remaining same as before.

Final Phase-3: (voice command)

i) At this stage, we converted the whole program go to Raspberry Pi. At first we copied the script and install dependencies on the Raspberry Pi (pip install pyniryo2 speech recognition sounddevice scipy pyttsx3 faster-whisper, plus any OS-level audio packages like portaudio19-dev).

ii) Then we plug in the USB microphone and pair the bluetooth speaker. Then set it as the default audio output so sound-device/pyttsx3 play through it without extra configuration in the script.

iii) We made a systemd service (which lets the script start automatically on the boot and keep running without a monitor, keyboard or manual login.

iv) For executing this, we first create a service unit file e.g. /etc/systemd/system/voicecontrol.service, describing:

→ ExecStart → the full command to run the script (e.g: python3 /home/pi/voicecommandfinal.py)

→ working directory — the folder containing the script

→ user — which system user runs it.

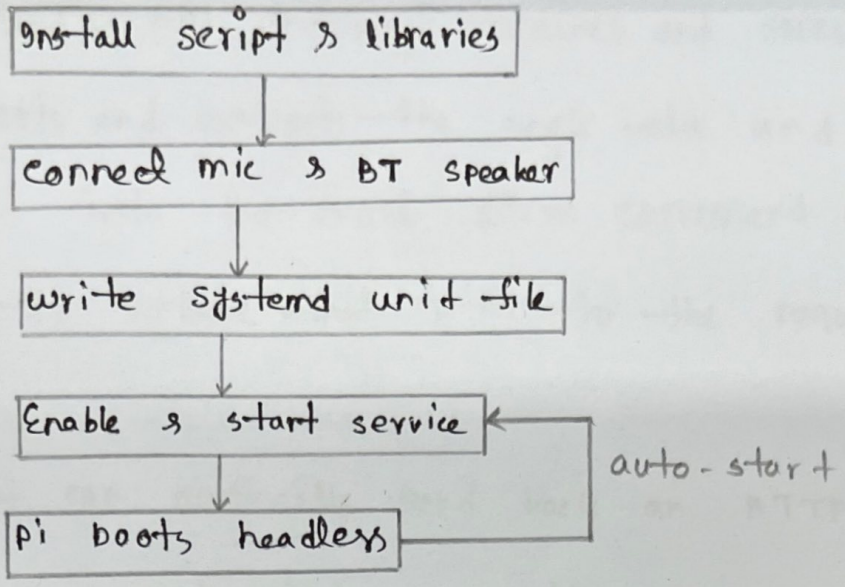
→ Restart=on-failure — automatically restarts the script if it crashes

→ WantedBy=multi-user.target — tells systemd to start this service during normal multi-user boot, before any desktop/login screen.

iv) we reloaded systemd's configuration (sudo systemctl daemon-reload) so it's set to start automatically on every future boot

v) for starting it immediately (sudo systemctl start voicecontrol.service) to test it without rebooting.

vi) Then we reboot the pi to confirm it comes up and starting listening on its own, with no monitor or keyboard attached — check remotely via sudo systemctl status voicecontrol.service over SSH.



Controlling an external prototype Hand Robot:

i) The prototype hand robot's controller runs a small web server and listens for http requests on a specific IP address and port,

ii) Our script/program builds a small JSON payload containing the the target joint angle.

iii) This payload is sent to the hand robot using either:

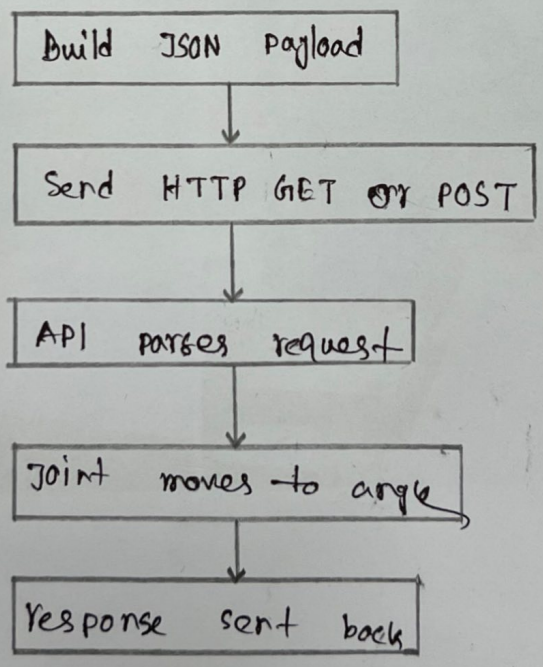
→ GET - the angle value is included in the URL as a query parameter

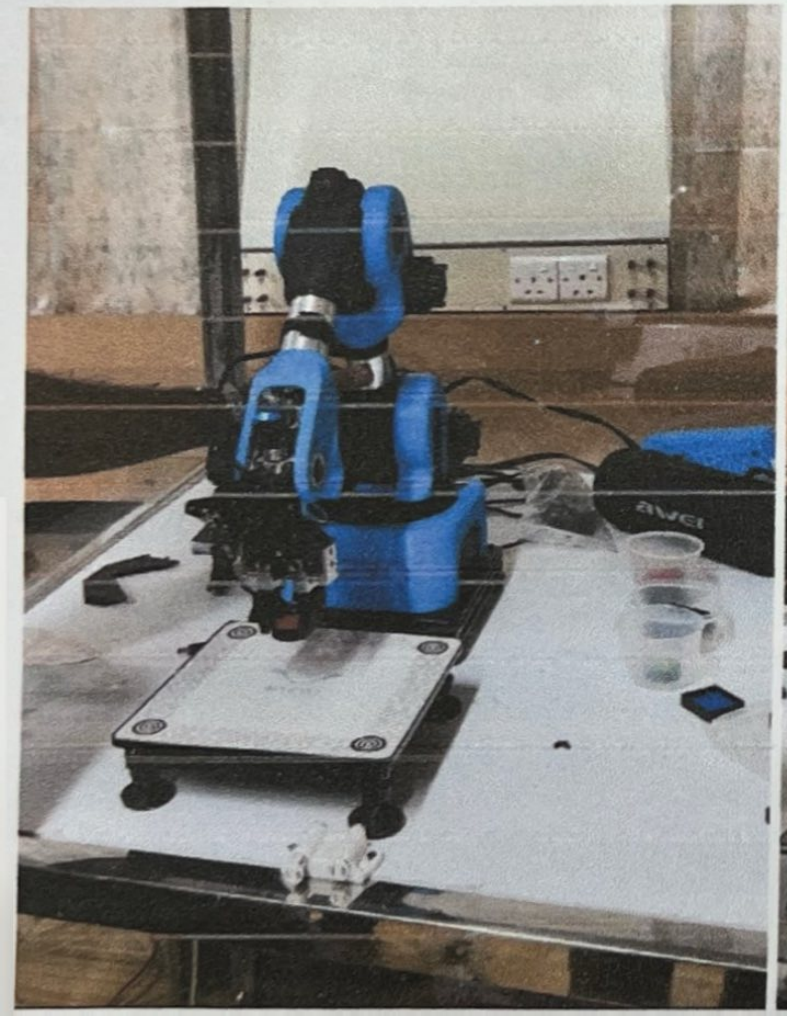
→ POST - the json payload is send in the request body with a JOSN, typically used when sending more structured or longer data.

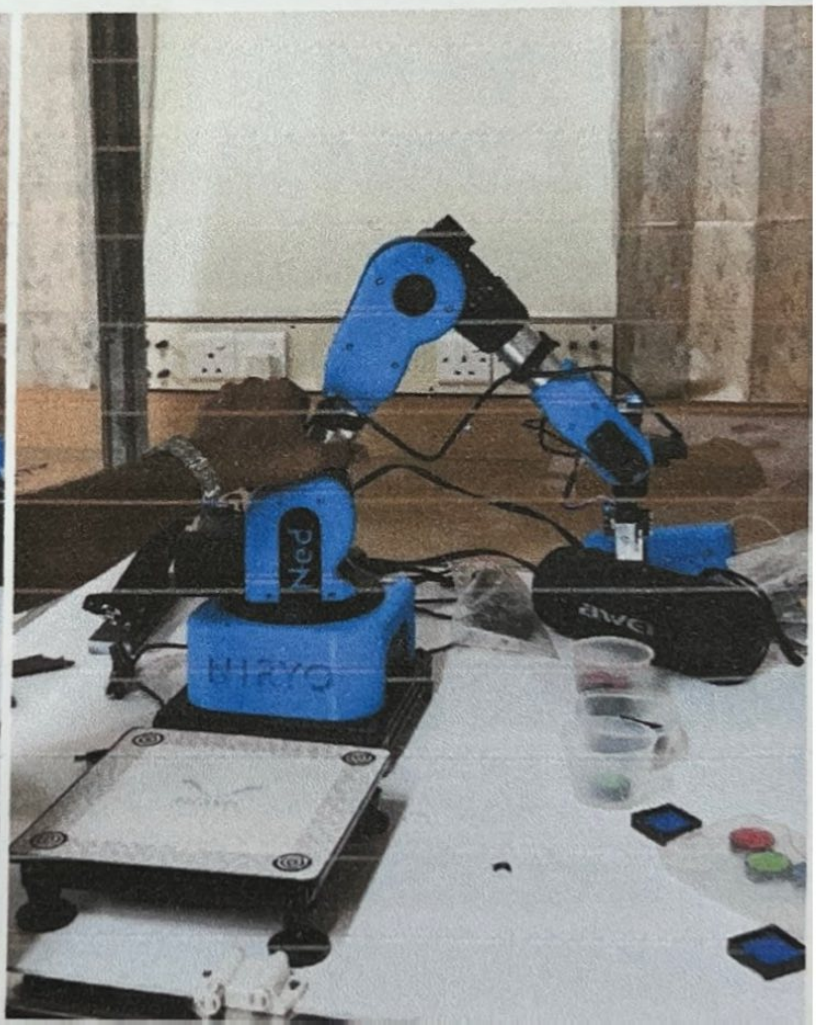
iv) The hand robot's API endpoint receives and pares the incoming requests and extracts the angle value and internally converts it into the exact servo command needed to physically to rotate that joint to the requested degree.

v) The hand robot can optinoally send back an HTTP response confirming the succeeded, which our side can use to verify the command was received correctly.

Visualize:







Conclusion:

This project demonstrated that a single robotic platform - the Niryo Ned - can be controlled through multiple, purpose - matched methods rather than one fixed interface. The autono - mous vision pick loop ran directly on the robot's own on board computer for unattended, repetitive sorting. Voice control was offloaded to a low-power Raspberry Pi 5 so the robot could be commanded hands-free and headlessly, without needing a monitor or constant supervision. Real-time gesture control ran on a laptop, since it was the only platform with enough compute power to handle MediaPipe's hand tracking model along side a live phone-camera feed. Finally, the project was extended beyond the Niryo Ned itself by integrating with an external prototype hand robot over a simple http/JSON API, showing that the same joint-level control concept can be reused across entirely different hardware.